



EECS 453

Principles of

Machine Learning

Lecture 6 – Supporter Vector Machines

Instructor: Prof. Qing Qu

Lecture Agenda

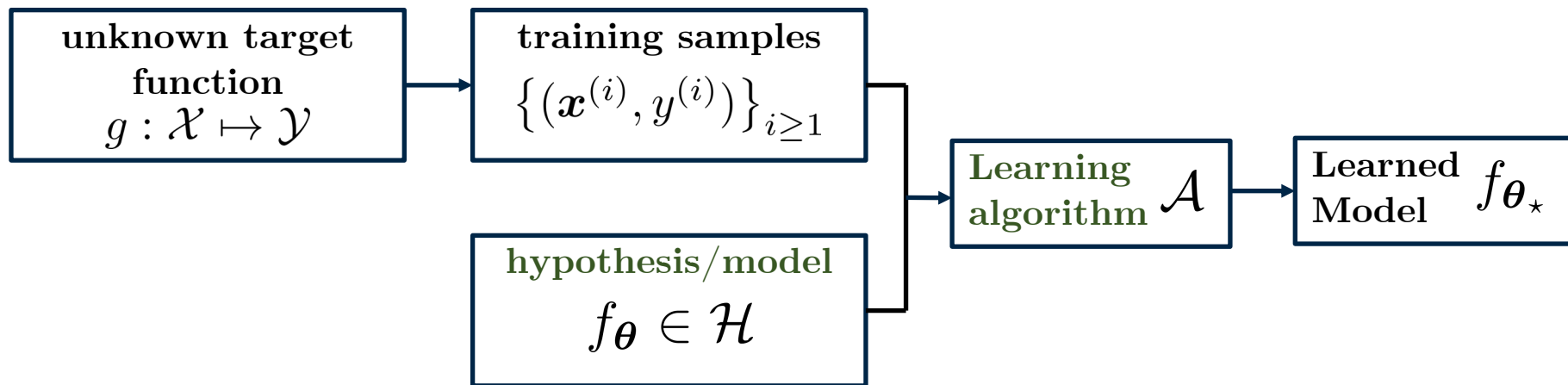
- Linear Classification Revisit
- Margin & Max-Margin Classifier
- Support Vector Machine (SVM)
 - Hard-Margin SVM
 - Soft-Margin SVM

Lecture Agenda

- **Linear Classification Revisit**
- Margin & Max-Margin Classifier
- Support Vector Machine (SVM)
 - Hard-Margin SVM
 - Soft-Margin SVM

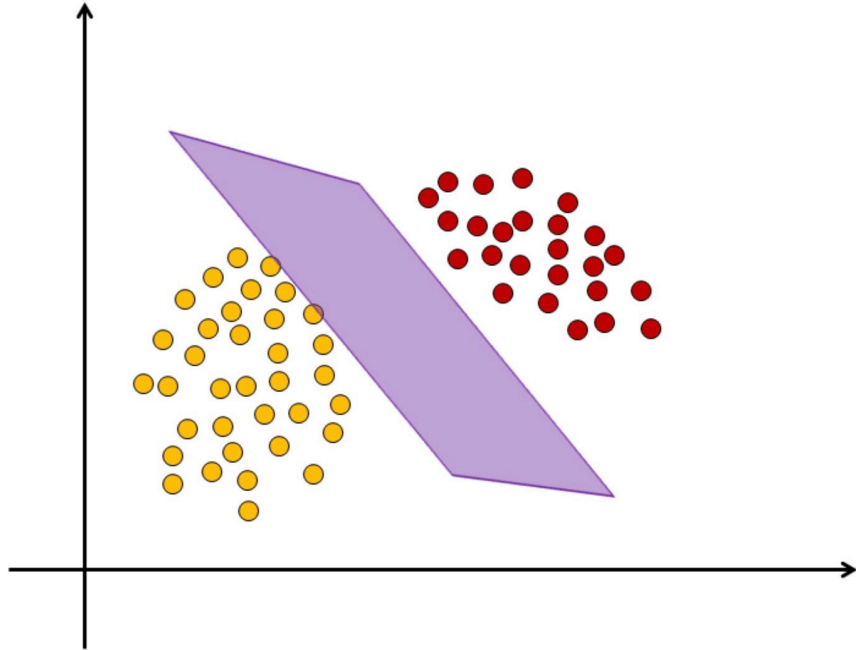
Supervised Learning

Seen pictorially, the process is therefore like the following



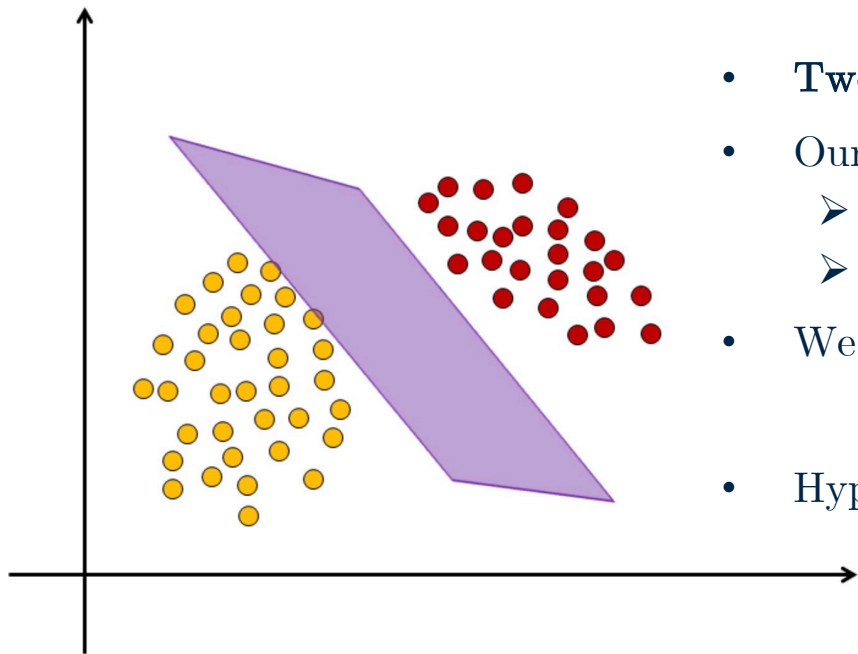
Supervised Classification Problem

The goal of supervised classification is to construct a **decision boundary** such that the two classes can be (maximally) separated.



Supervised Classification Problem

The goal of supervised classification is to construct a **decision boundary** such that the two classes can be (maximally) **separated**.



- Two classes: $\mathcal{Y} = \{-1, 1\}$ or $\mathcal{Y} = \{0, 1\}$, no difference
- Our goal is to learn to classify two types of examples:
 - Class 0 – labeled as 0 or -1
 - Class 1 – labeled as 1
- We would like to learn $f : \mathcal{X} \mapsto \{-1, 1\}$, such that
- Hypothesis $h : \mathcal{X} \mapsto \mathcal{Y}$. Ideally, we would like

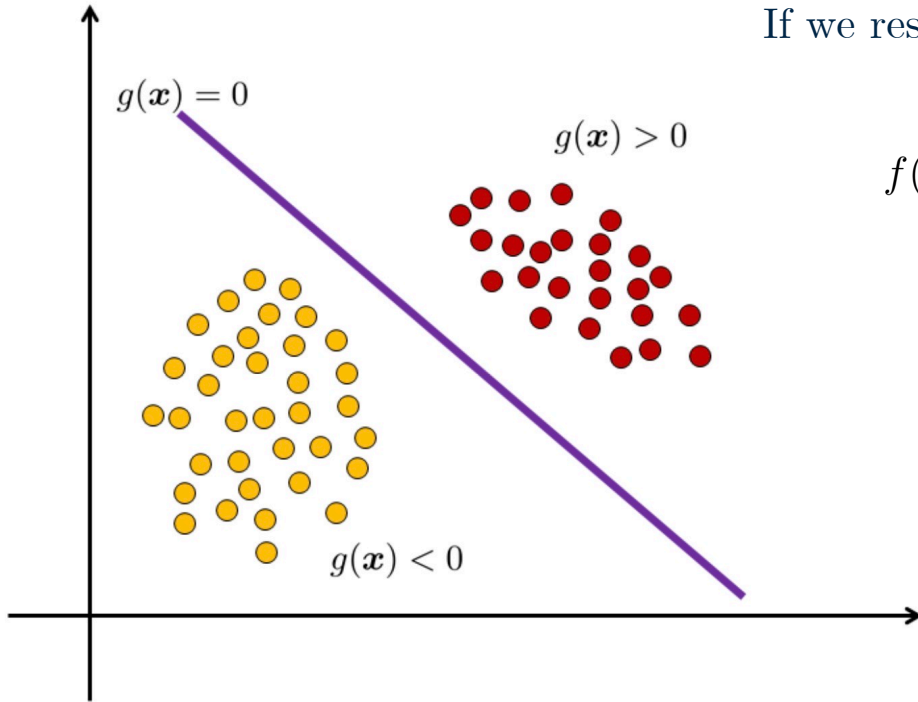
$$y_n = f(\mathbf{x}_n)$$

$$h(\mathbf{x}) \approx f(\mathbf{x}), \quad \forall \mathbf{x} \in \mathcal{X}.$$

Classification: the Binary Case

If we restrict ourselves to **binary classifier**, then

$$f(\mathbf{x}) = \begin{cases} 1 & \text{if } g(\mathbf{x}) > 0 \\ 0 & \text{if } g(\mathbf{x}) < 0 \\ \text{either} & \text{if } g(\mathbf{x}) = 0 \end{cases}$$

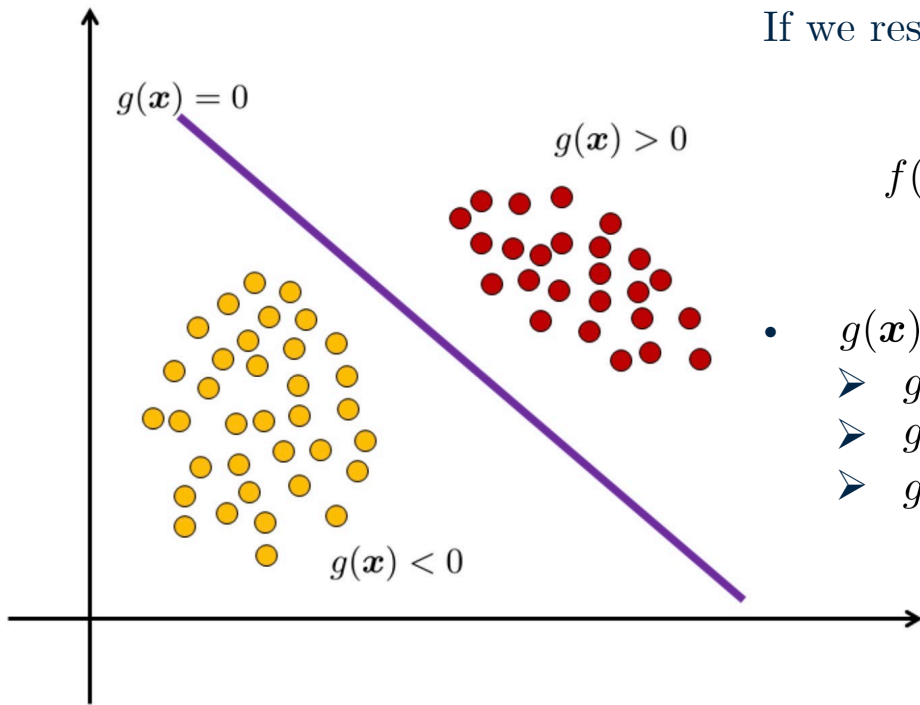


Classification: the Binary Case

If we restrict ourselves to **binary classifier**, then

$$f(\mathbf{x}) = \begin{cases} 1 & \text{if } g(\mathbf{x}) > 0 \\ 0 & \text{if } g(\mathbf{x}) < 0 \\ \text{either} & \text{if } g(\mathbf{x}) = 0 \end{cases}$$

- $g(\mathbf{x})$ is called a **discriminant function**:
 - $g(\mathbf{x}) > 0$: $\mathbf{x}$ lives on the **positive** side of $g(\cdot)$
 - $g(\mathbf{x}) < 0$: $\mathbf{x}$ lives on the **negative** side of $g(\cdot)$
 - $g(\mathbf{x}) = 0$: the **decision boundary**.



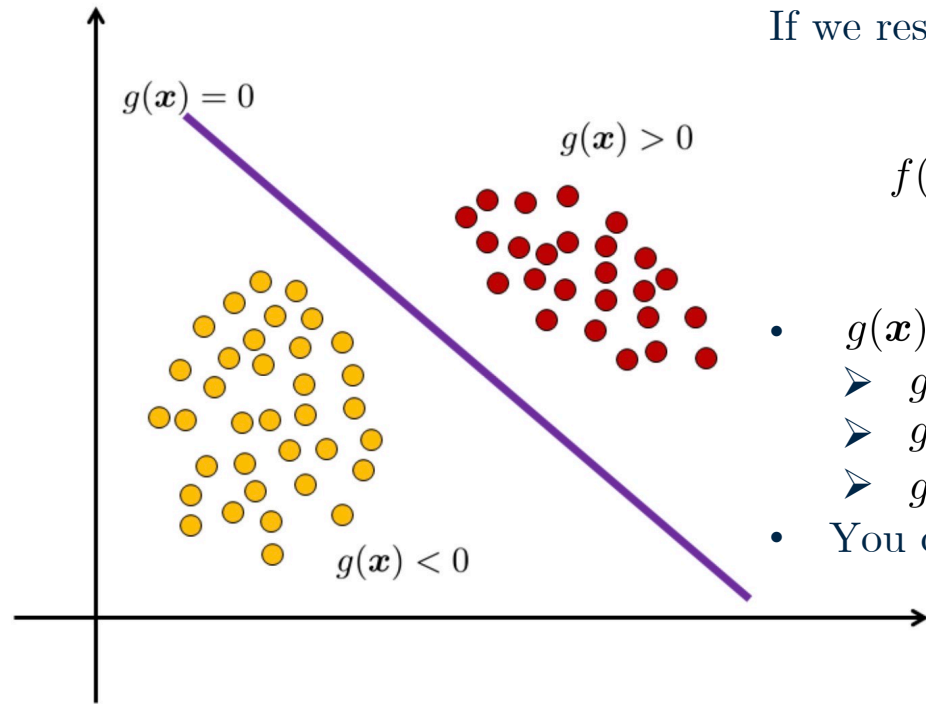
Classification: the Binary Case

If we restrict ourselves to **binary classifier**, then

$$f(\mathbf{x}) = \begin{cases} 1 & \text{if } g(\mathbf{x}) > 0 \\ 0 & \text{if } g(\mathbf{x}) < 0 \\ \text{either} & \text{if } g(\mathbf{x}) = 0 \end{cases}$$

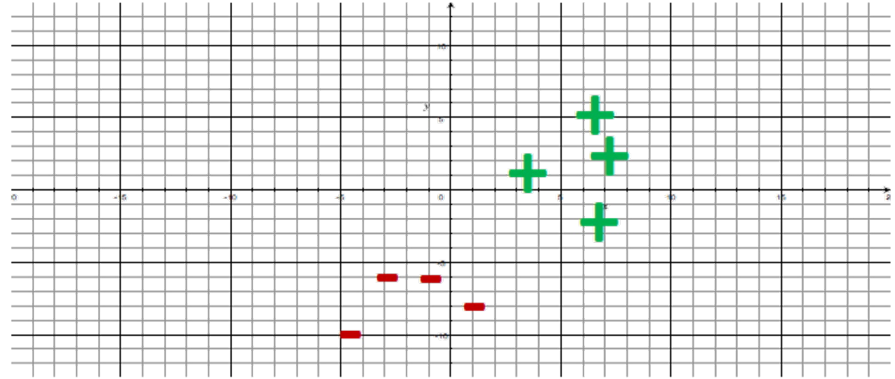
- $g(\mathbf{x})$ is called a **discriminant function**:
 - $g(\mathbf{x}) > 0$: $\mathbf{x}$ lives on the **positive** side of $g(\cdot)$
 - $g(\mathbf{x}) < 0$: $\mathbf{x}$ lives on the **negative** side of $g(\cdot)$
 - $g(\mathbf{x}) = 0$: the **decision boundary**.
- You can also claim: (no difference)

$$f(\mathbf{x}) = \text{sign}(g(\mathbf{x})) = \begin{cases} 1 & \text{if } g(\mathbf{x}) > 0 \\ -1 & \text{if } g(\mathbf{x}) < 0 \\ \text{either} & \text{if } g(\mathbf{x}) = 0 \end{cases}$$

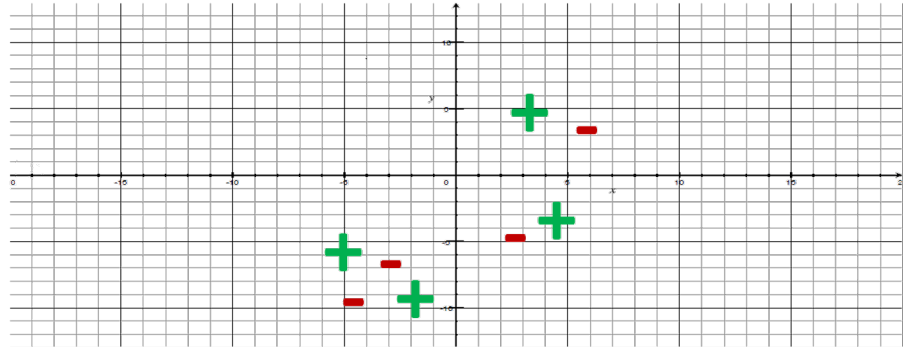


Linear Separability: Intuition

linear separable



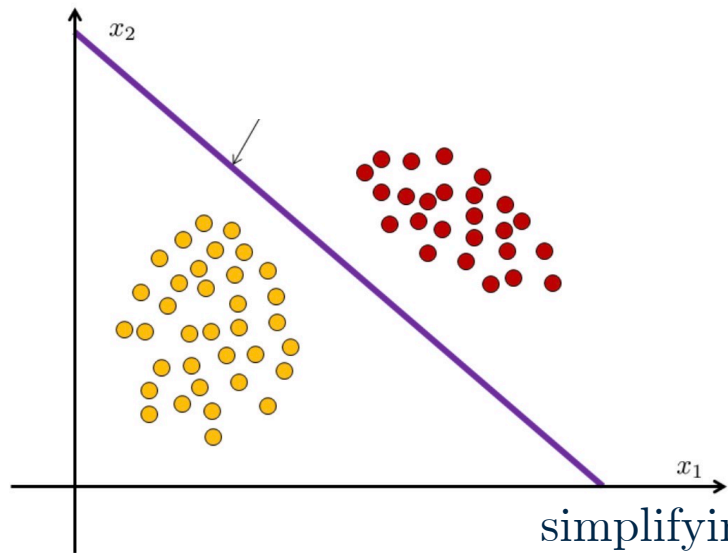
not linearly separable



Linear Classifier

Given: training data $S_n = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i \in \mathbb{R}^d, y_i \in \mathbb{R}$

Goal: Learn a *linear decision boundary* (through the origin) that minimizes *training error*



simplifying assumptions: linear separability

$$f(\mathbf{x}; \boldsymbol{\theta}) = \text{sign}(g(\mathbf{x})) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$$

$$\begin{aligned} \text{Err}_n(\boldsymbol{\theta}) &= \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{y_i \neq f(\mathbf{x}_i, \boldsymbol{\theta})} \\ &= \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{y_i \cdot (\mathbf{w}^\top \mathbf{x}_i + b) \leq 0} \end{aligned}$$

Linear Discriminant Function

A *linear discriminant function* takes the form

$$g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$

- $\mathbf{w} \in \mathbb{R}^d$: linear coefficients
- $b \in \mathbb{R}$: bias/offset
- Define the overall parameter: $\boldsymbol{\theta} = \{\mathbf{w}, b\} \in \mathbb{R}^{d+1}$

$$g(\mathbf{x}) = \boldsymbol{\theta}^\top \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}$$

Linear Discriminant Function

A *linear discriminant function* takes the form

$$g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$

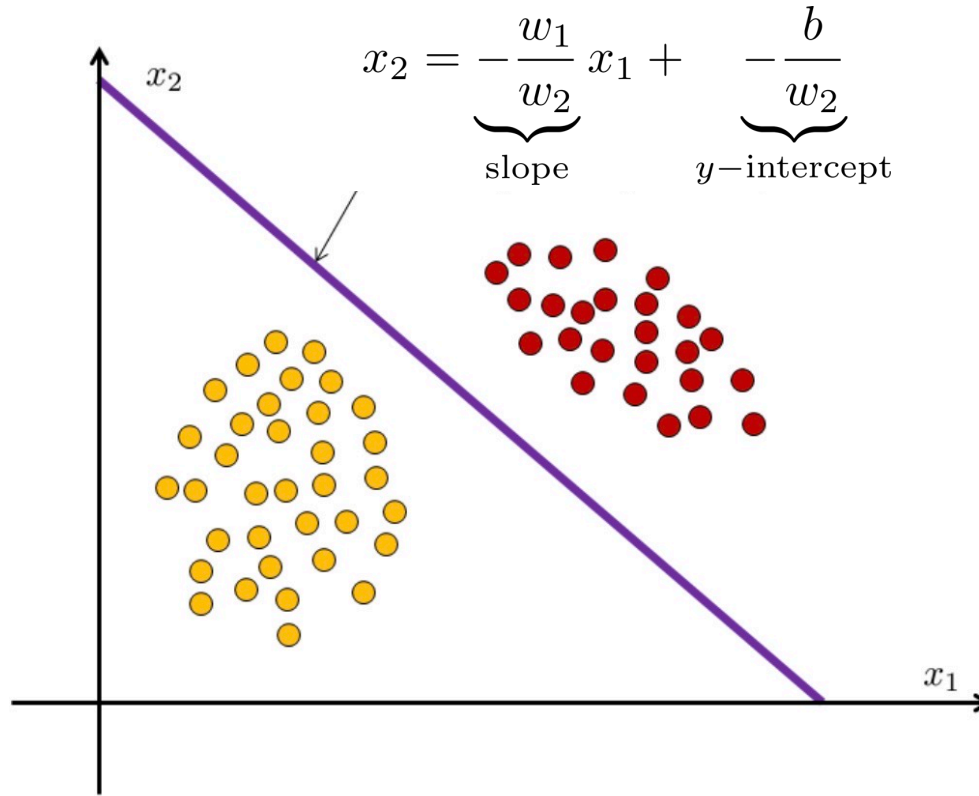
- $\mathbf{w} \in \mathbb{R}^d$: linear coefficients
- $b \in \mathbb{R}$: bias/offset
- Define the overall parameter: $\boldsymbol{\theta} = \{\mathbf{w}, b\} \in \mathbb{R}^{d+1}$

$$g(\mathbf{x}) = \boldsymbol{\theta}^\top \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}$$

Example:

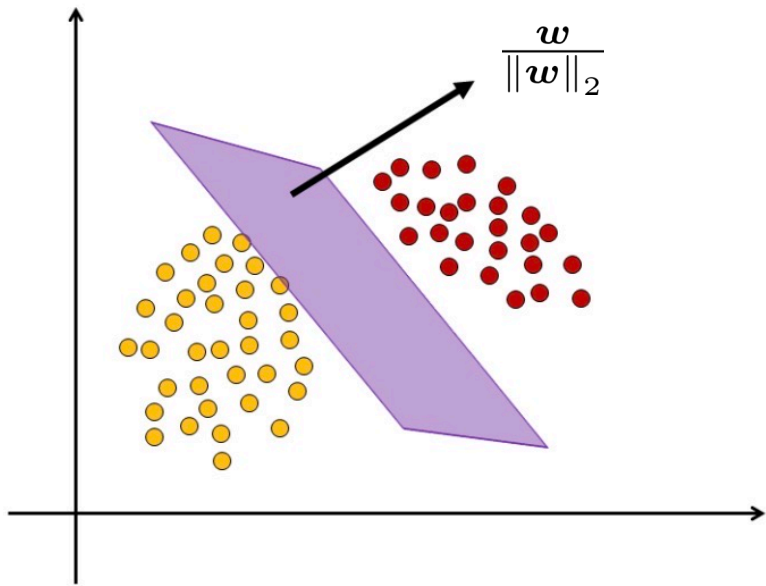
- If $d = 2$, then $g(\mathbf{x}) = w_2 x_2 + w_1 x_1 + b$
- $g(\mathbf{x}) = 0$ implies that
$$x_2 = \underbrace{-\frac{w_1}{w_2} x_1}_{\text{slope}} + \underbrace{-\frac{b}{w_2}}_{y\text{-intercept}}$$

Linear Discriminant Function



Linear Discriminant Function

- In high-dimension $\mathbf{x} \in \mathbb{R}^d$,
 $g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ is a **hyperplane**



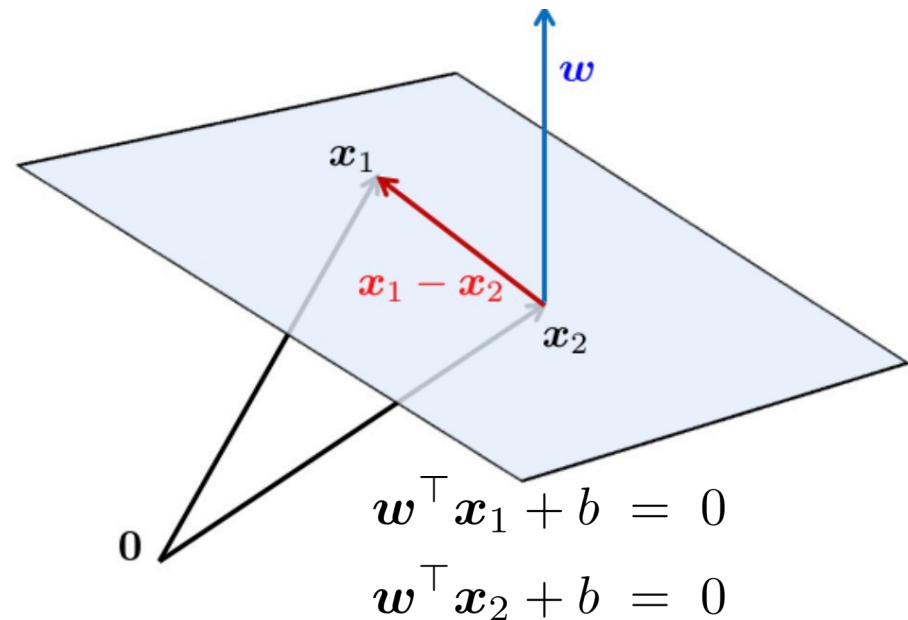
- Separating hyperplane:**

$$\begin{aligned}\mathcal{H} &:= \{\mathbf{x} \in \mathbb{R}^d \mid g(\mathbf{x}) = 0\} \\ &= \{\mathbf{x} \in \mathbb{R}^d \mid \mathbf{w}^\top \mathbf{x} + b = 0\}\end{aligned}$$

- $\mathbf{x} \in \mathcal{H}$ means $\mathbf{x} \in \mathbb{R}^d$ is on the decision boundary
- $\frac{\mathbf{w}}{\|\mathbf{w}\|_2}$ is the **normal vector** of $\mathcal{H}$.

Linear Discriminant Function

- In high-dimension $\mathbf{x} \in \mathbb{R}^d$,
 $g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ is a **hyperplane**
- $\mathbf{w} \in \mathbb{R}^d$ is the **normal vector**
of the hyperplane $\mathcal{H}$

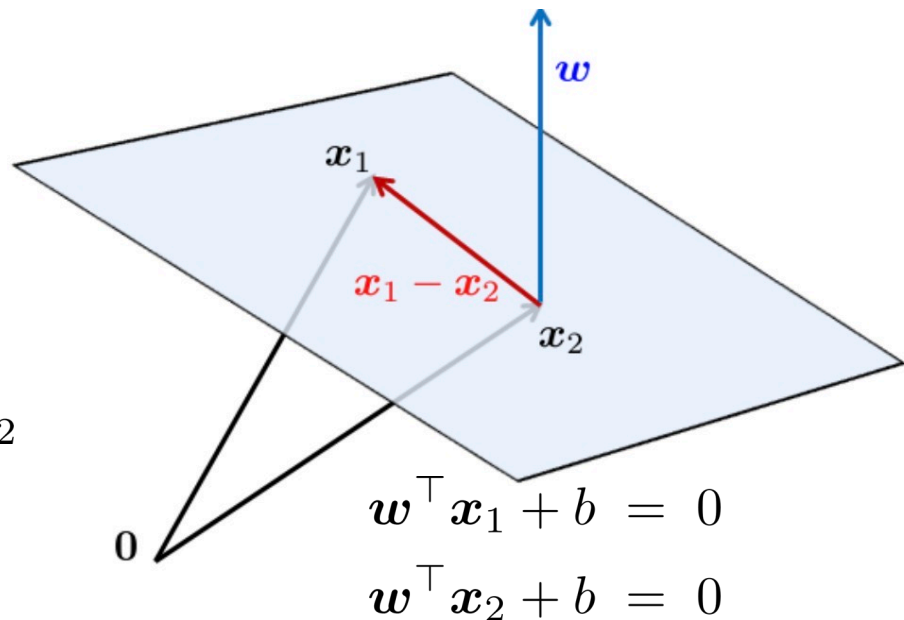


Linear Discriminant Function

- In high-dimension $\mathbf{x} \in \mathbb{R}^d$,
 $g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ is a **hyperplane**
- $\mathbf{w} \in \mathbb{R}^d$ is the **normal vector**
of the hyperplane $\mathcal{H}$

$$\mathbf{w}^\top (\mathbf{x}_1 - \mathbf{x}_2) = 0$$

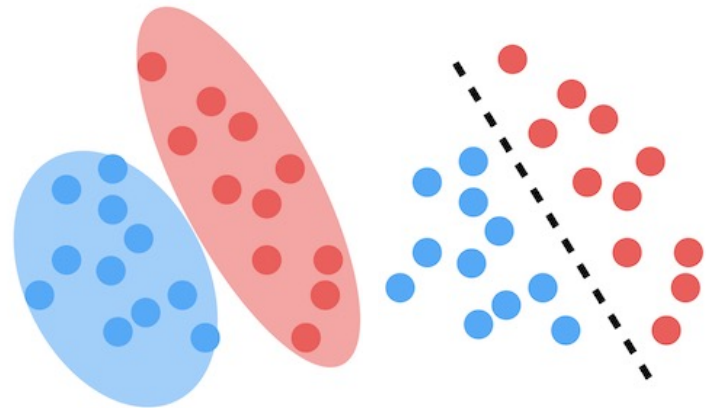
- $\mathbf{x}_1 - \mathbf{x}_2$ is a **tangent** vector
of the hyperplane $\mathcal{H}$
- $\mathbf{w}$ is *perpendicular* to $\mathbf{x}_1 - \mathbf{x}_2$
and hence **normal**



Generative vs. Discriminative Methods

Both **LDA** and **logistic regression** classifiers are **linear classifiers**

estimating the posterior $p_{y|x}(k|\mathbf{x})$



Generative vs. Discriminative Methods

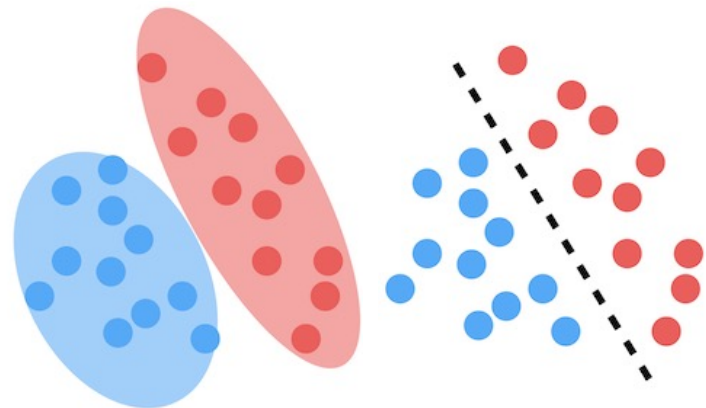
Both **LDA** and **logistic regression** classifiers are **linear classifiers**

estimating the posterior $p_{y|x}(k|\mathbf{x})$

- **Linear discriminant analysis:**

- Based on Gaussian generative models
- Needs to estimate the joint distribution of $(\mathbf{x}, y)$ from the data

$$\mathbf{x} \mid y = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}), \quad \text{for } k = 0, 1, \dots, K - 1$$



Generative vs. Discriminative Methods

Both **LDA** and **logistic regression** classifiers are **linear classifiers**

estimating the posterior $p_{y|x}(k|\mathbf{x})$

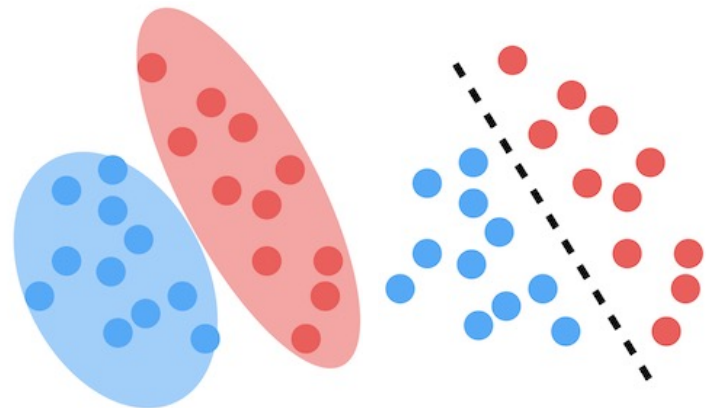
- **Linear discriminant analysis:**

- Based on Gaussian generative models
- Needs to estimate the joint distribution of $(\mathbf{x}, y)$ from the data

$$\mathbf{x} \mid y = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}), \quad \text{for } k = 0, 1, \dots, K - 1$$

$$\mathbf{w} = \hat{\boldsymbol{\Sigma}}^{-1} (\hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_0) \in \mathbb{R}^d$$

$$b = \frac{1}{2} \hat{\boldsymbol{\mu}}_0^\top \hat{\boldsymbol{\Sigma}}^{-1} \hat{\boldsymbol{\mu}}_0 - \frac{1}{2} \hat{\boldsymbol{\mu}}_1^\top \hat{\boldsymbol{\Sigma}}^{-1} \hat{\boldsymbol{\mu}}_1 + \log \frac{\hat{p}_y(1)}{\hat{p}_y(0)}$$



Generative vs. Discriminative Methods

Both **LDA** and **logistic regression** classifiers are **linear classifiers**

estimating the posterior $p_{y|x}(k|\mathbf{x})$

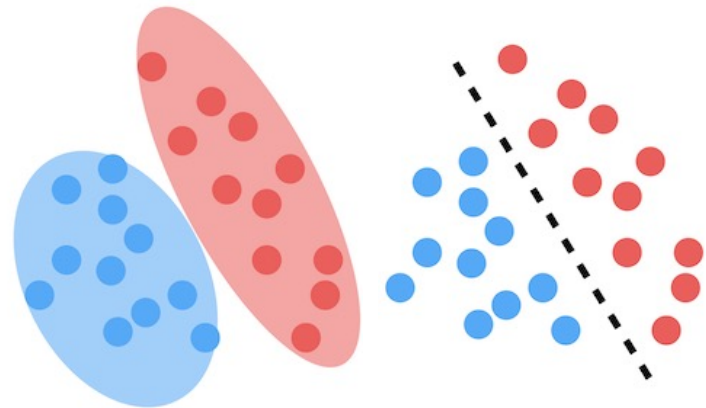
➤ Logistic regression:

- Model only the distribution of $y \mid \mathbf{x}$, not $(\mathbf{x}, y)$

$$p_{y|\mathbf{x}}(y = 1 \mid \mathbf{x}) = \frac{1}{1 + \exp(-(\mathbf{w}^\top \mathbf{x} + b))}$$

- Valid for a larger class of distributions $(\mathbf{x}, y)$
- $\boldsymbol{\theta} = (\mathbf{w}, b)$ are estimated via solving the logistic regression problem

$$\min_{\boldsymbol{\theta}} -\mathcal{L}(\boldsymbol{\theta}) = -\sum_{i=1}^n [(y_i - 1) \boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i - \log(1 + \exp(-\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i))]$$

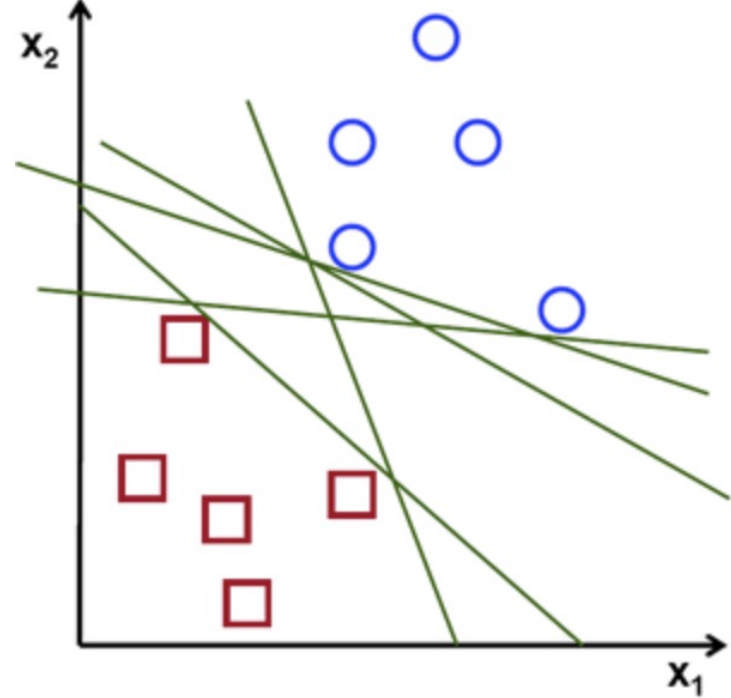


Lecture Agenda

- Linear Classification Revisit
- **Margin & Max-Margin Classifier**
- Support Vector Machine (SVM)
 - Hard-Margin SVM
 - Soft-Margin SVM

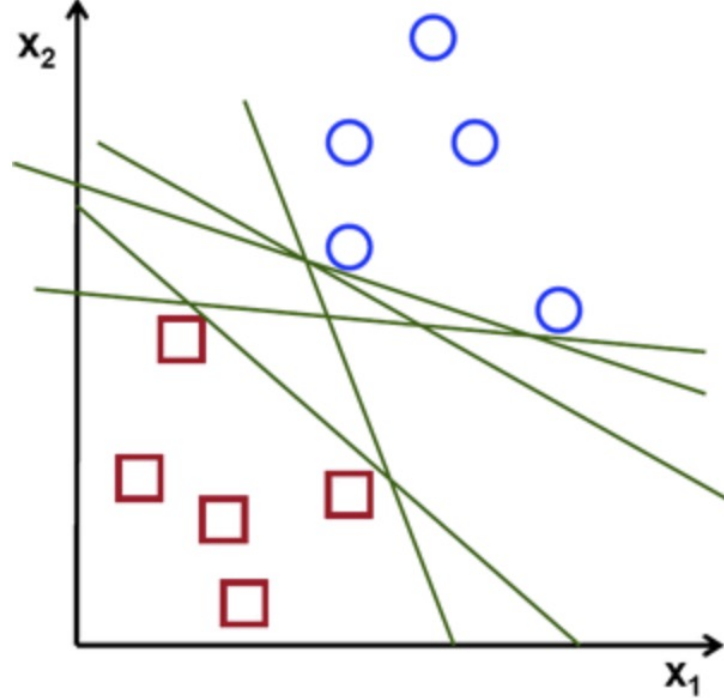
the Best Linear Classifier?

- Given the data are linearly separable, there could be **many** lines that separate the two classes...



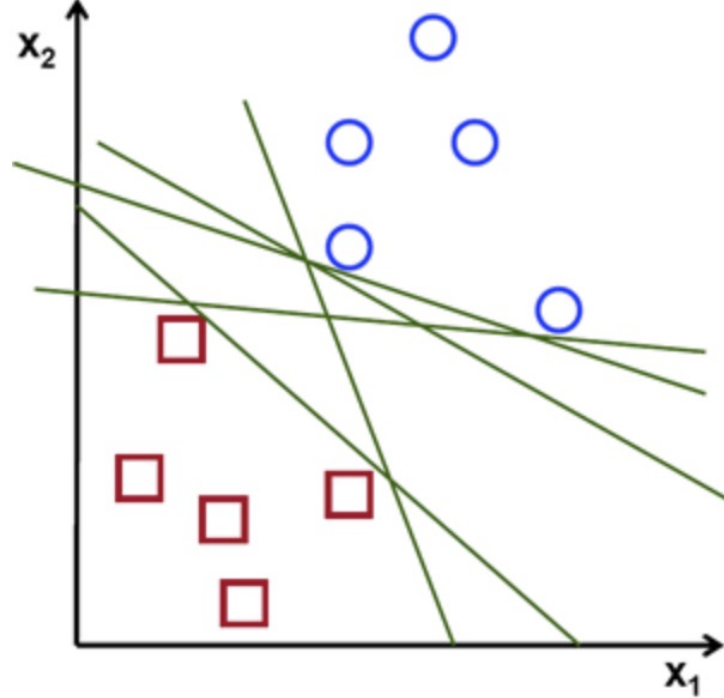
the Best Linear Classifier?

- Given the data are linearly separable, there could be **many** lines that separate the two classes...
- There could be **multiple** solutions $(\mathbf{w}, b)$ for LDA and logistic regression

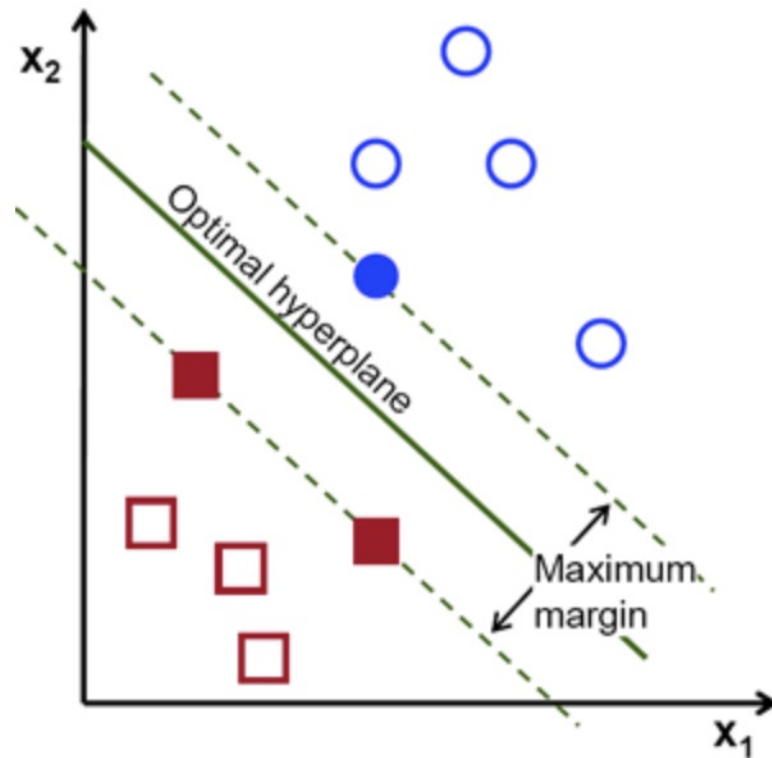
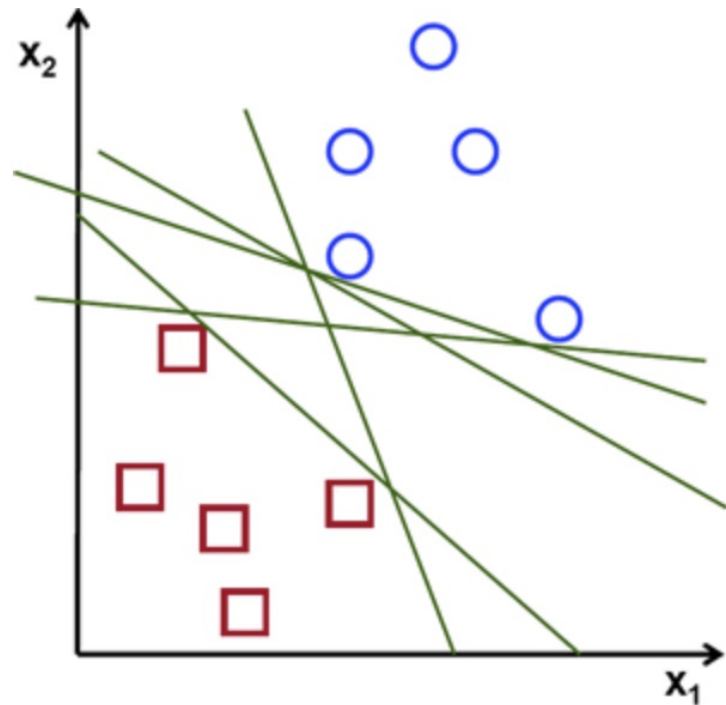


the Best Linear Classifier?

- Given the data are linearly separable, there could be **many** lines that separate the two classes...
- There could be **multiple** solutions $(\mathbf{w}, b)$ for LDA and logistic regression
- Is there a principle that gives you the **best** linear classifier?

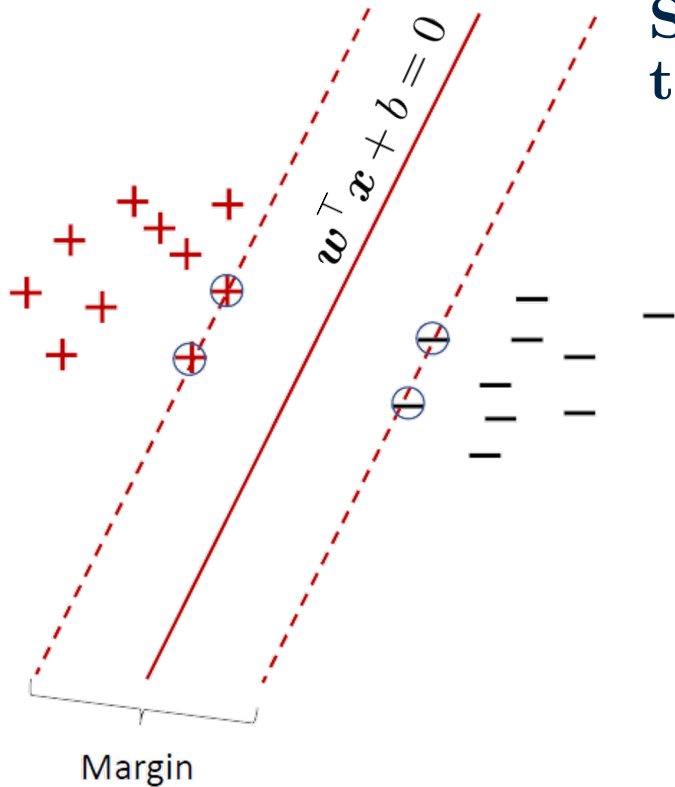


the Best Linear Classifier?

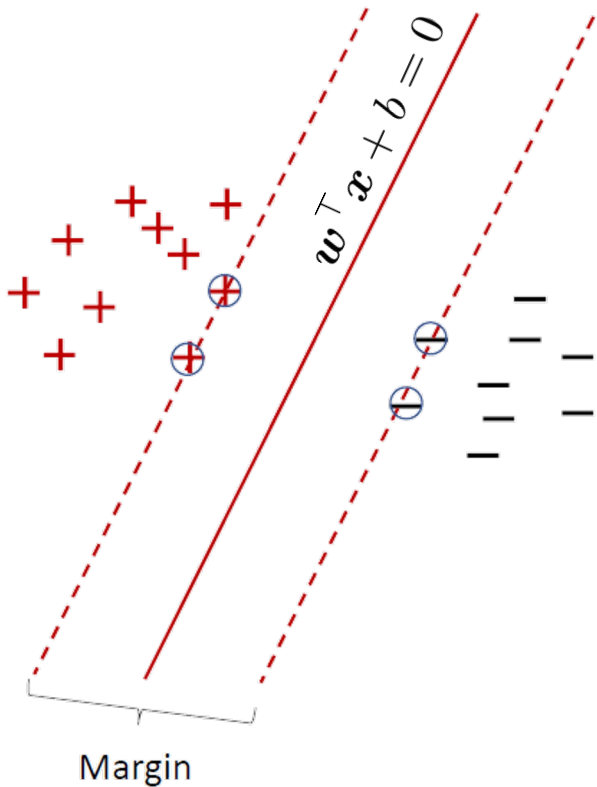


Maximal Margin and Support Vectors

Solution: find the linear classifier with the largest margin!



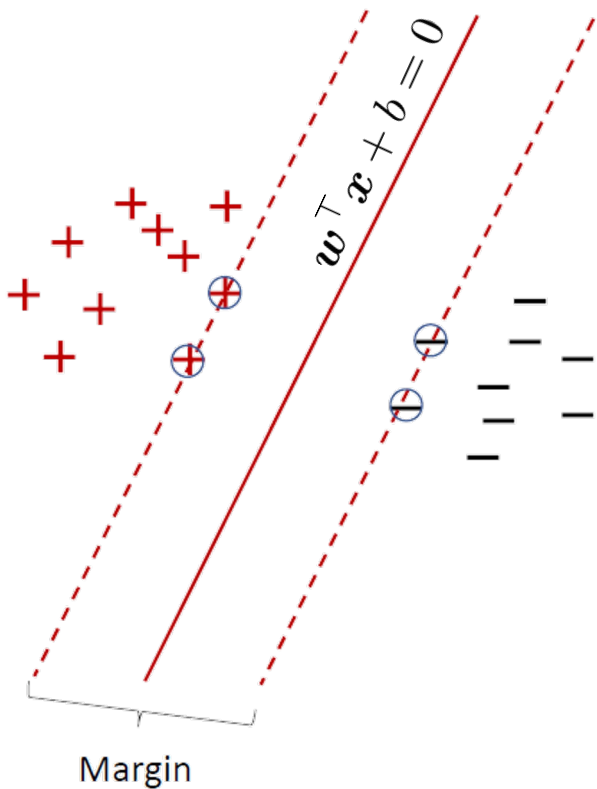
Maximal Margin and Support Vectors



Solution: find the linear classifier with the largest margin!

- Select **two** parallel hyperplanes that separate the two classes and the distance between them is *as large as possible*.

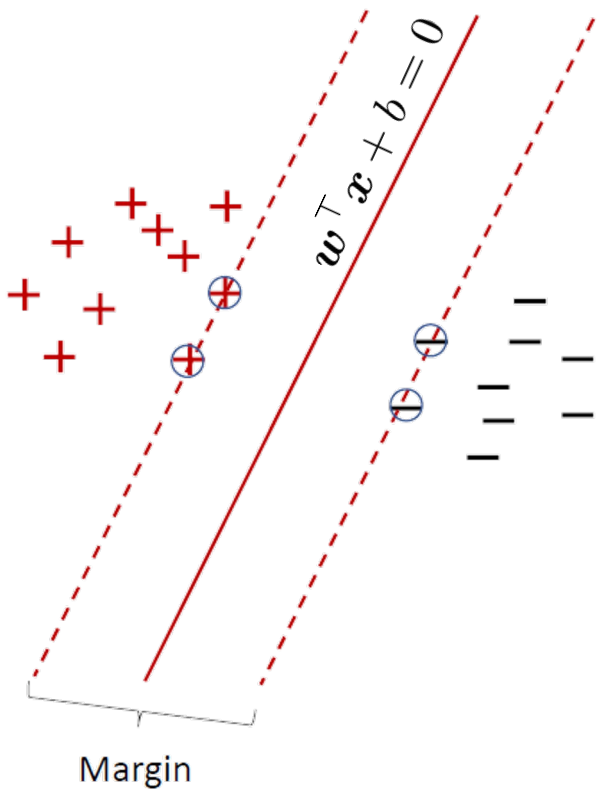
Maximal Margin and Support Vectors



Solution: find the linear classifier with the largest margin!

- Select **two** parallel hyperplanes that separate the two classes and the distance between them is *as large as possible*.
- **Support vectors:** points that determine the two parallel hyperplanes.

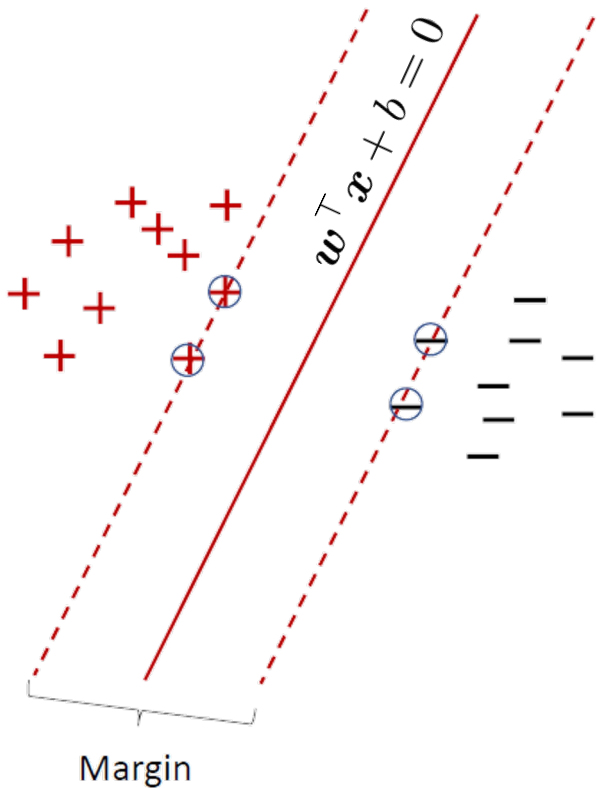
Maximal Margin and Support Vectors



Solution: find the linear classifier with the largest margin!

- Select **two** parallel hyperplanes that separate the two classes and the distance between them is *as large as possible*.
- **Support vectors:** points that determine the two parallel hyperplanes.
- **Margin:** the region bounded by these two hyperplanes

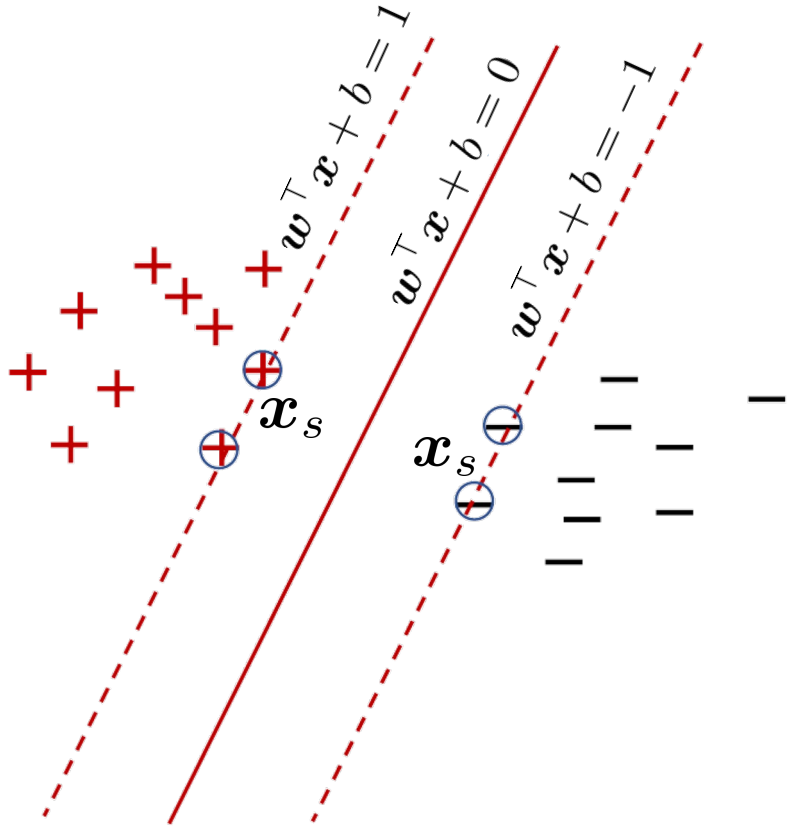
Maximal Margin and Support Vectors



Solution: find the linear classifier with the largest margin!

- Select **two** parallel hyperplanes that separate the two classes and the distance between them is *as large as possible*.
- **Support vectors:** points that determine the two parallel hyperplanes.
- **Margin:** the region bounded by these two hyperplanes
- The *maximum-margin hyperplane* lies halfway between them, given by $w^T x + b = 0$

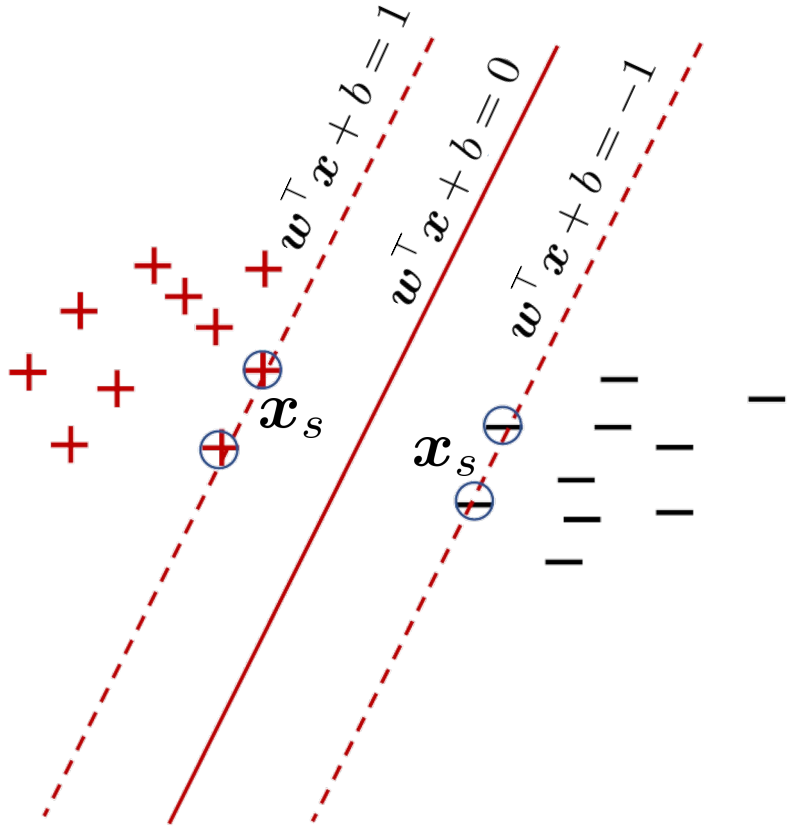
Expression of the Two Parallel Hyperplanes



- **Support vectors:** $\{x_s\}$
- Two parallel hyperplanes passes the support vectors:

$$w^\top x_s + b = C$$

Expression of the Two Parallel Hyperplanes



- **Support vectors:** $\{x_s\}$
- Two parallel hyperplanes passes the support vectors:

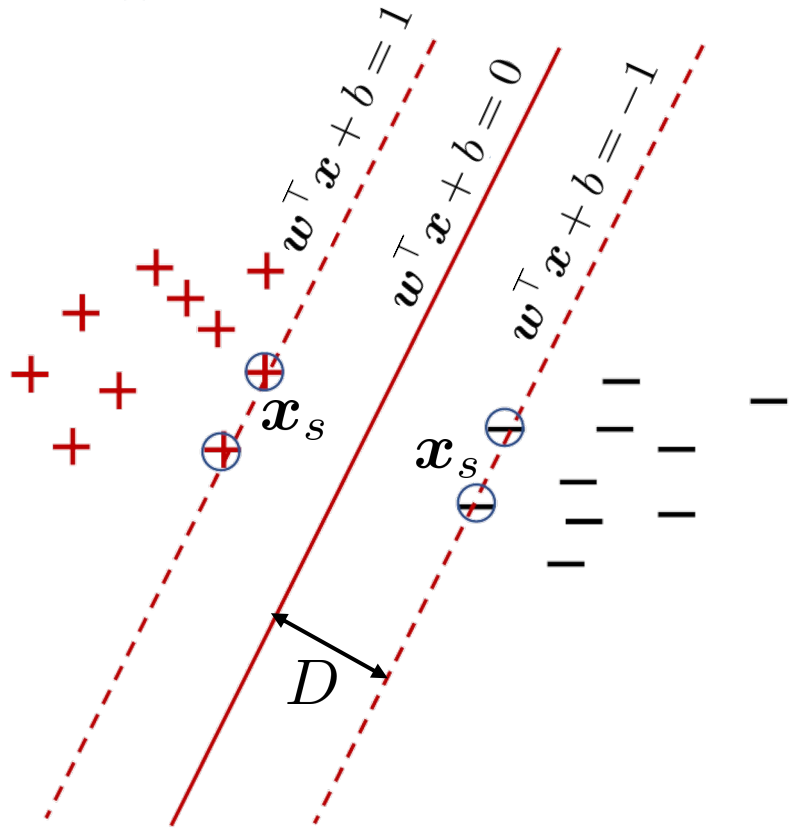
$$w^\top x_s + b = C$$

- We can do **normalization** to $\theta = (w, b)$, so that

$$w^\top x_s + b = 1$$

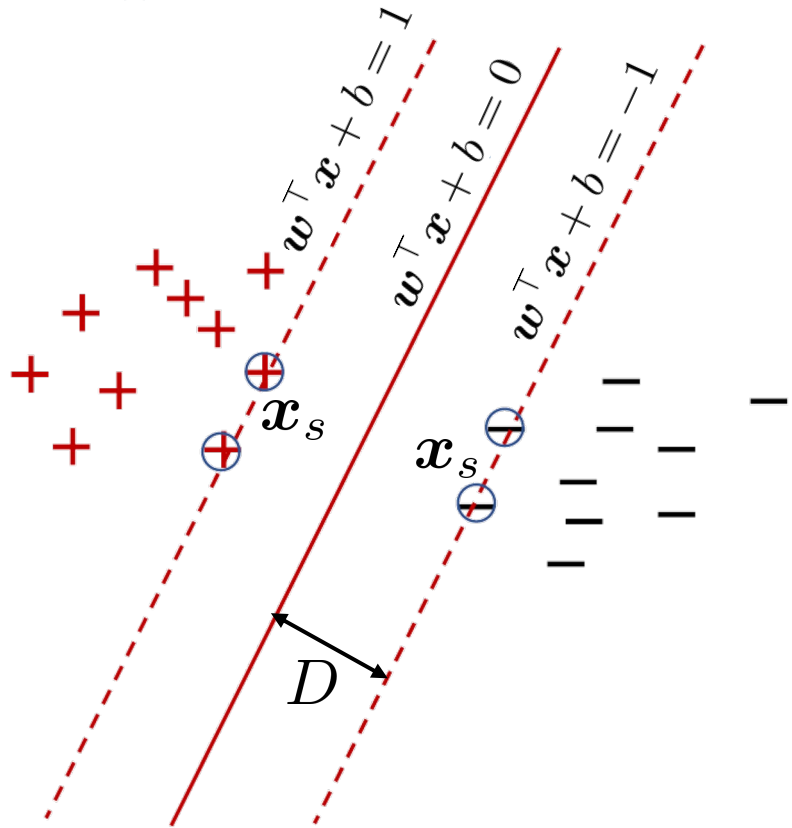
$$w^\top x_s + b = -1$$

Expression of the Margin



- D : the distance between the maximum-margin hyperplane and x_s
- **Margin** = $2 \times D$

Expression of the Margin



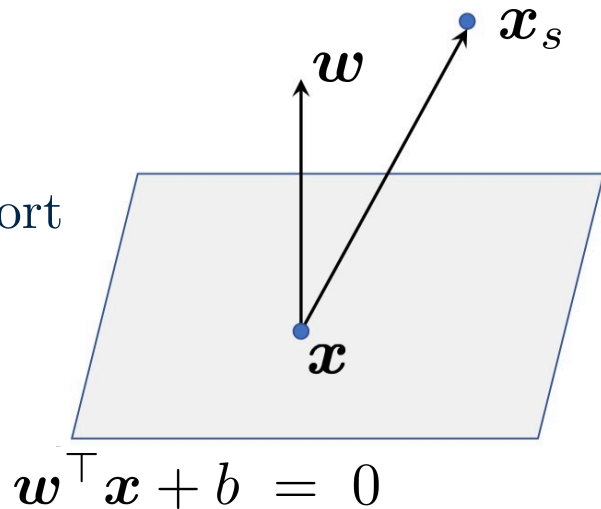
- D : the distance between the maximum-margin hyperplane and x_s
- **Margin** $= 2 \times D$
- **Question:** how to calculate such a distance D ?

Expression of the Margin

- **Task:** calculate the distance D between the support vector $\mathbf{x}_s$ and the maximum-margin hyperplane

$$\mathcal{H} = \{\mathbf{x} \mid \mathbf{w}^\top \mathbf{x} + b = 0\}$$

given that $|\mathbf{w}^\top \mathbf{x}_s + b| = 1$

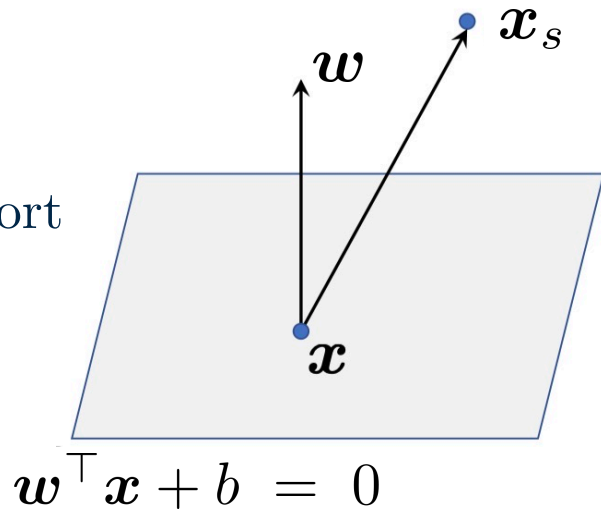


Expression of the Margin

- **Task:** calculate the distance D between the support vector $\mathbf{x}_s$ and the maximum-margin hyperplane

$$\mathcal{H} = \{ \mathbf{x} \mid \mathbf{w}^\top \mathbf{x} + b = 0 \}$$

given that $|\mathbf{w}^\top \mathbf{x}_s + b| = 1$



- **Fact:** $\mathbf{w}$ is the normal vector of $\mathcal{H}$.

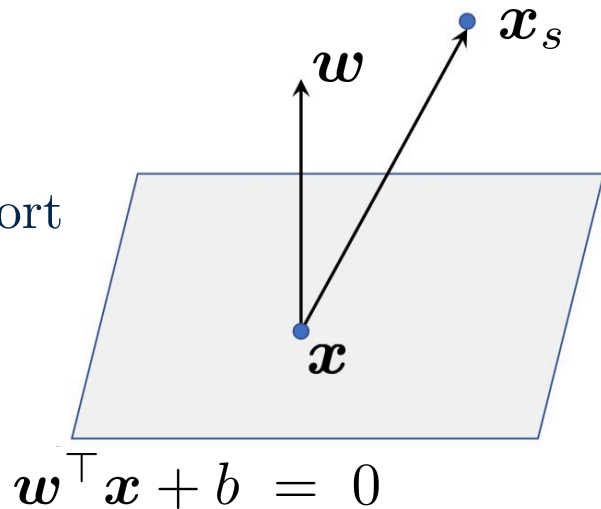
$$\mathbf{w}^\top (\mathbf{x}' - \mathbf{x}'') = 0, \quad \forall \mathbf{x}', \mathbf{x}'' \in \mathcal{H}.$$

Expression of the Margin

- **Task:** calculate the distance D between the support vector $\mathbf{x}_s$ and the maximum-margin hyperplane

$$\mathcal{H} = \{ \mathbf{x} \mid \mathbf{w}^\top \mathbf{x} + b = 0 \}$$

given that $|\mathbf{w}^\top \mathbf{x}_s + b| = 1$



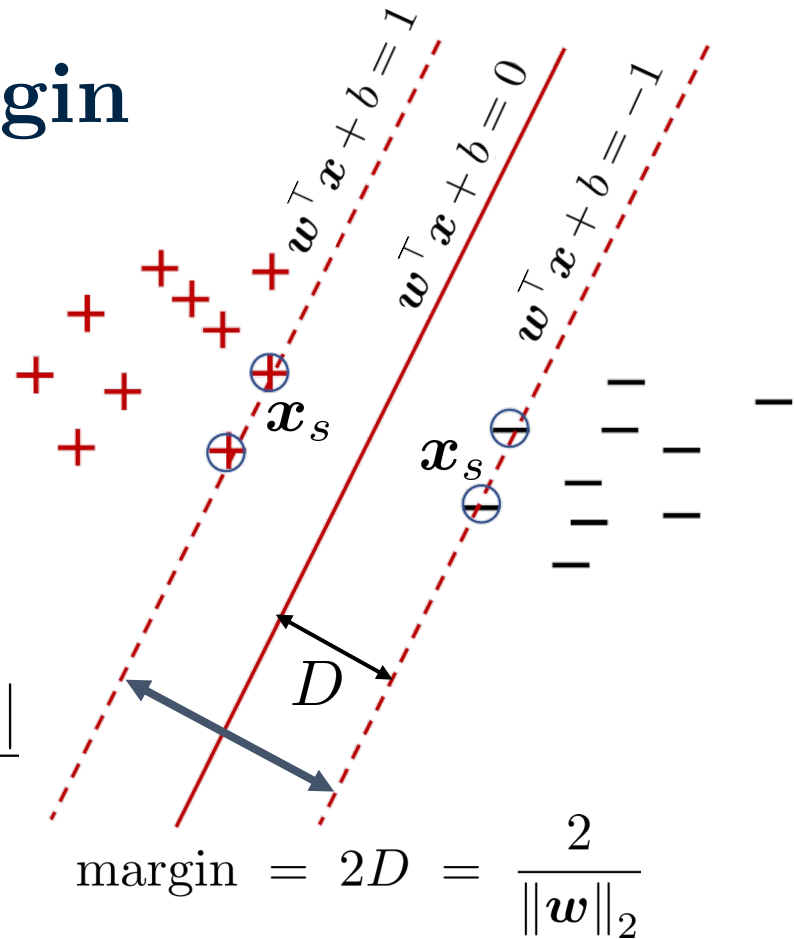
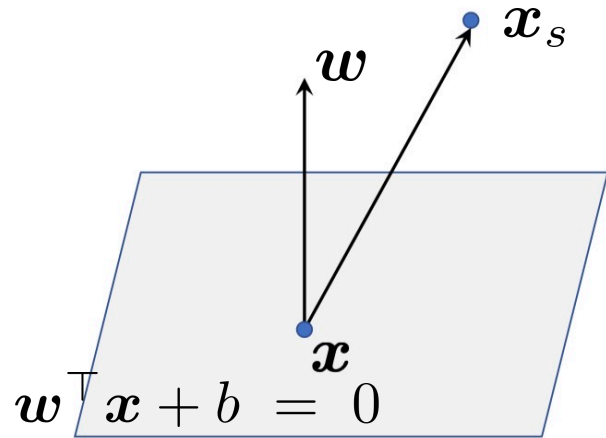
- **Fact:** $\mathbf{w}$ is the normal vector of $\mathcal{H}$.

$$\mathbf{w}^\top (\mathbf{x}' - \mathbf{x}'') = 0, \quad \forall \mathbf{x}', \mathbf{x}'' \in \mathcal{H}.$$

- **Solution:** the distance D equals to the projection $\mathbf{x}_s - \mathbf{x}$ of onto $\mathbf{w}$:

$$D = \frac{\mathbf{w}^\top (\mathbf{x}_s - \mathbf{x})}{\|\mathbf{w}\|_2}.$$

Expression of the Margin



- Finally, the margin

$$\begin{aligned} 2 \frac{|w^\top (x_s - x)|}{\|w\|_2} &= 2 \frac{|(w^\top x_s + b) - (w^\top x + b)|}{\|w\|_2} \\ &= \frac{2}{\|w\|_2}. \end{aligned}$$

Lecture Agenda

- Linear Classification Revisit
- Margin & Max-Margin Classifier
- **Support Vector Machine (SVM)**
 - **Hard-Margin SVM**
 - Soft-Margin SVM

Learning a Max-Margin Classifier

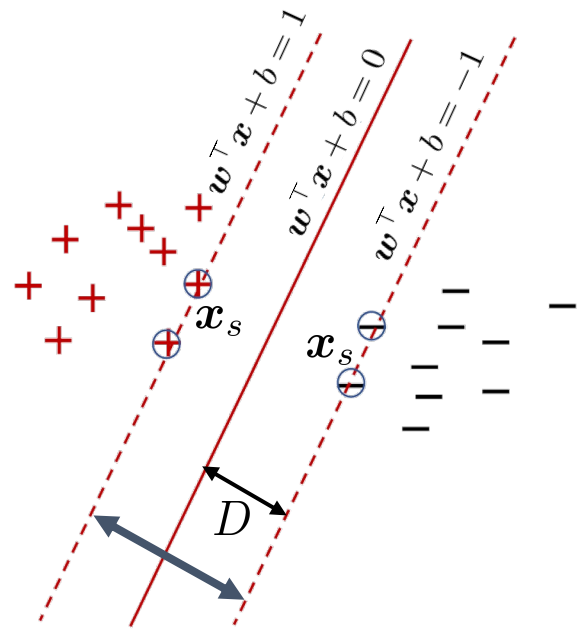
- **The principle:** maximize the margin

$$\max_{\mathbf{w} \in \mathbb{R}^d} \frac{2}{\|\mathbf{w}\|_2}$$

which is *equivalent* to

$$\min_{\mathbf{w} \in \mathbb{R}^d} \|\mathbf{w}\|_2^2$$

- **Question:** do we miss anything?



$$\text{margin} = 2D = \frac{2}{\|\mathbf{w}\|_2}$$

Learning a Max-Margin Classifier

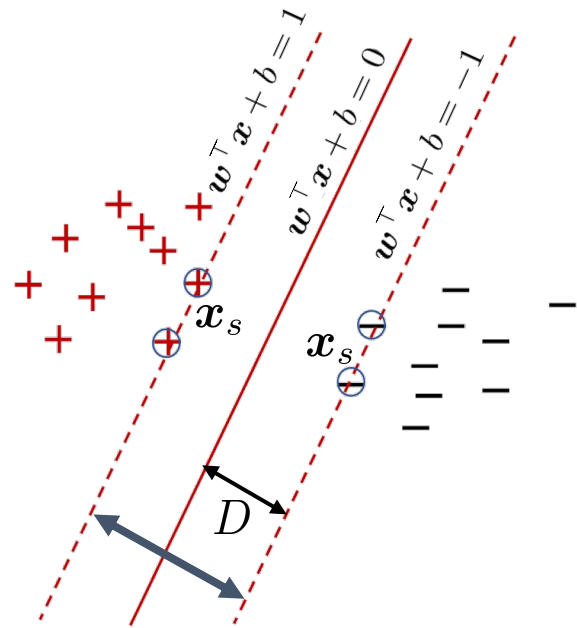
- The problem is *unbounded*, we forgot a constraint:

$$|\mathbf{w}^\top \mathbf{x}_s + b| = 1.$$

Thus, we have the problem

$$\min_{\mathbf{w} \in \mathbb{R}^d} \|\mathbf{w}\|_2^2$$

$$\text{s.t. } |\mathbf{w}^\top \mathbf{x}_i + b| \geq 1, \quad i = 1, \dots, n$$



$$\text{margin} = 2D = \frac{2}{\|\mathbf{w}\|_2}$$

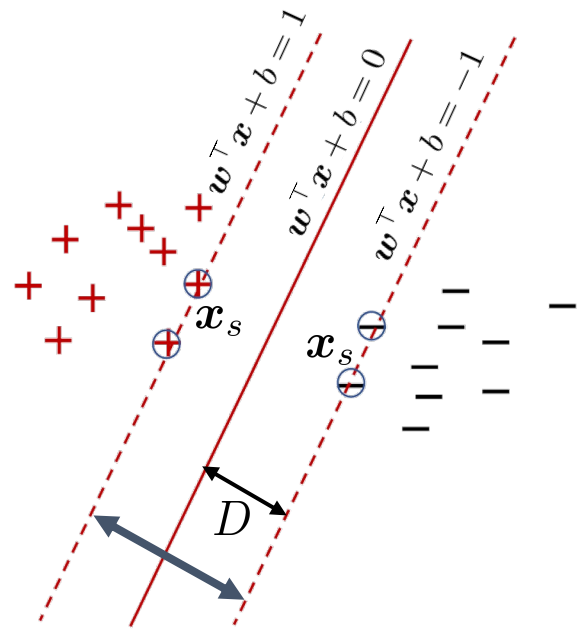
Hard-Margin SVM

- Finally, since

$$|\mathbf{w}^\top \mathbf{x}_s + b| = y_i (\mathbf{w}^\top \mathbf{x}_i + b)$$

we have the optimization formulation for the **hard-margin SVM** as

$$\begin{aligned} \min_{\mathbf{w} \in \mathbb{R}^d} \quad & \|\mathbf{w}\|_2^2 \\ \text{s.t.} \quad & y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$



$$\text{margin} = 2D = \frac{2}{\|\mathbf{w}\|_2}$$

Solving Hard-Margin SVM

$$\begin{aligned} \min_{\mathbf{w} \in \mathbb{R}^d} \mathcal{L}(\mathbf{w}) &= \|\mathbf{w}\|_2^2 \\ \text{s.t. } y_i (\mathbf{w}^\top \mathbf{x}_i + b) &\geq 1, \quad i = 1, \dots, n \end{aligned}$$

- This is a constrained **quadratic programming (QP)** problem, which is a **convex problem**.
- If we can find the optimal solution $(\mathbf{w}_\star, b_\star)$, we can make the inference for an input $\mathbf{x}$ by

$$\hat{y} = \text{sign}(\mathbf{w}_\star^\top \mathbf{x} + b_\star)$$

Solving Hard-Margin SVM

$$\begin{aligned} \min_{\mathbf{w} \in \mathbb{R}^d} \mathcal{L}(\mathbf{w}) &= \|\mathbf{w}\|_2^2 \\ \text{s.t. } y_i (\mathbf{w}^\top \mathbf{x}_i + b) &\geq 1, \quad i = 1, \dots, n \end{aligned}$$

- Gradient descent:

$$\mathbf{w}^{k+1} = \mathbf{w}^k - \tau \cdot \nabla \mathcal{L}(\mathbf{w}^k)$$

Solving Hard-Margin SVM

$$\begin{aligned} \min_{\mathbf{w} \in \mathbb{R}^d} \mathcal{L}(\mathbf{w}) &= \|\mathbf{w}\|_2^2 \\ \text{s.t. } y_i (\mathbf{w}^\top \mathbf{x}_i + b) &\geq 1, \quad i = 1, \dots, n \end{aligned}$$

- Gradient descent:

$$\mathbf{w}^{k+1} = \mathbf{w}^k - \tau \cdot \nabla \mathcal{L}(\mathbf{w}^k)$$

- Projection back to the constraints:

$$\mathbf{w}^{k+1} = \mathcal{P}_\Theta (\mathbf{w}^k - \tau \cdot \nabla \mathcal{L}(\mathbf{w}^k))$$

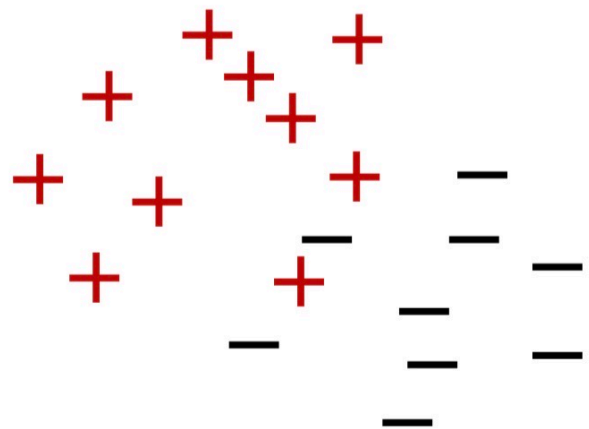
$$\Theta := \{ \mathbf{w} \in \mathbb{R}^d, b \mid y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n \}$$

The method is called **projected gradient descent**.

Lecture Agenda

- Linear Classification Revisit
- Margin & Max-Margin Classifier
- **Support Vector Machine (SVM)**
 - Hard-Margin SVM
 - **Soft-Margin SVM**

What if the Data is Not Linearly Separable?



$$\begin{aligned} \min_{\mathbf{w} \in \mathbb{R}^d} \mathcal{L}(\mathbf{w}) &= \|\mathbf{w}\|_2^2 \\ \text{s.t. } y_i (\mathbf{w}^\top \mathbf{x}_i + b) &\geq 1, \quad i = 1, \dots, n \end{aligned}$$

How can we handle such *more practical* cases?

- **Issue with hard-margin:** some points will never satisfy the margin constraint
- **Solution? Relaxing the constraints.**

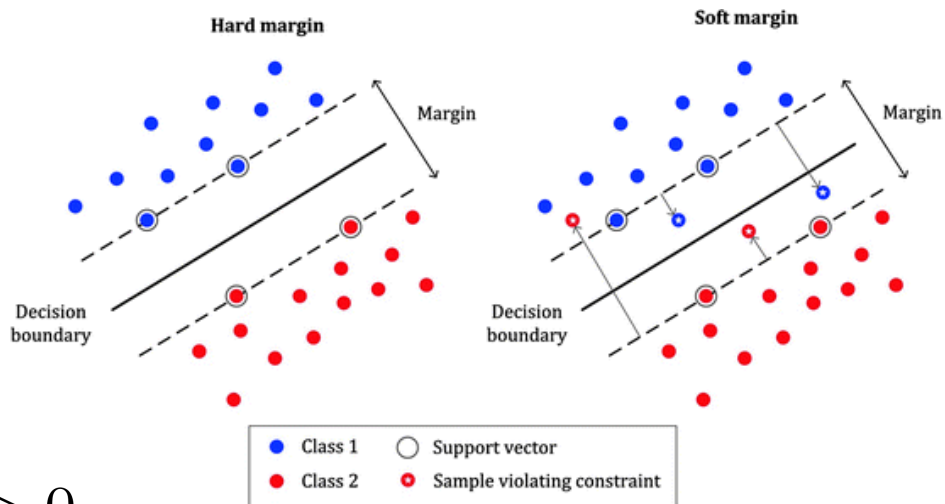
Soft-Margin SVM

$$\min_{\mathbf{w} \in \mathbb{R}^d, b, \boldsymbol{\xi} \in \mathbb{R}^n} \frac{\|\mathbf{w}\|_2^2}{2} + C \sum_{i=1}^n \xi_i$$

$$\text{s.t. } y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

$i = 1, \dots, n.$

soft constraints



- $\boldsymbol{\xi} \in \mathbb{R}^d$ is called the *slack variable*;
- $C > 0$ is a *penalty* (hyperparameter):
 - as $C \uparrow$, penalty on errors/misclassifications $\uparrow$
 - as $C \rightarrow \infty$, in the limit this is the hard margin SVM

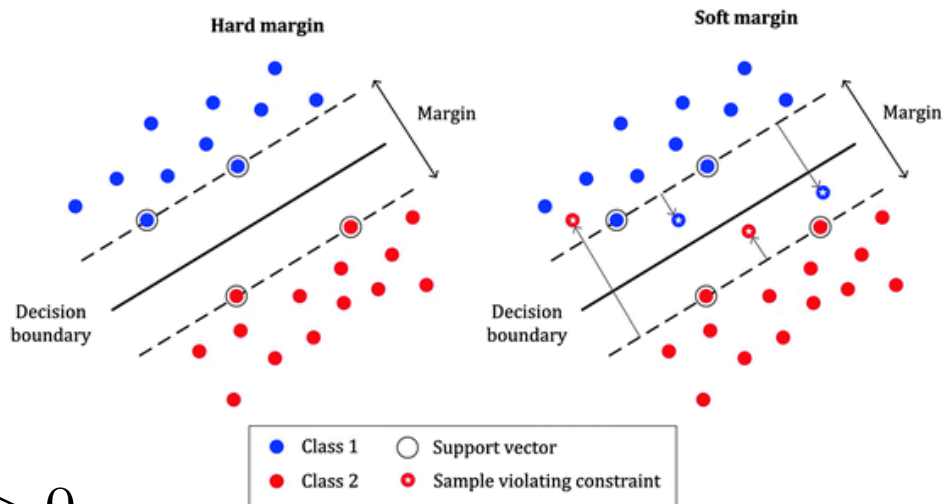
Soft-Margin SVM

$$\min_{\mathbf{w} \in \mathbb{R}^d, b, \boldsymbol{\xi} \in \mathbb{R}^n} \frac{\|\mathbf{w}\|_2^2}{2} + C \sum_{i=1}^n \xi_i$$

$$\text{s.t. } y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

$i = 1, \dots, n.$

soft constraints



Advantages of Soft-Margin SVM:

- can handle data that are not linearly separable
- reduce effect of outliers and less chances of overfitting

Soft-Margin SVM: Unconstrained Formulation

Fact: the constrained soft-margin SVM is equivalent to the following unconstrained formulation:

$$\min_{\mathbf{w} \in \mathbb{R}^d, b} \frac{1}{n} \sum_{i=1}^n \underbrace{\max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b))}_{H(\boldsymbol{\theta}; \mathbf{x}_i, y_i)} + \lambda \|\mathbf{w}\|_2^2$$

- The function $H(\boldsymbol{\theta}; \mathbf{x}_i, y_i)$ is called the ***Hinge loss***
 - $H(\boldsymbol{\theta}; \mathbf{x}_i, y_i) = 0$ if the point $\mathbf{x}_i$ lies in the correct side
 - For data on the wrong side, the functions value is proportional to the distance from the maximum-margin hyperplane.

Soft-Margin SVM: Unconstrained Formulation

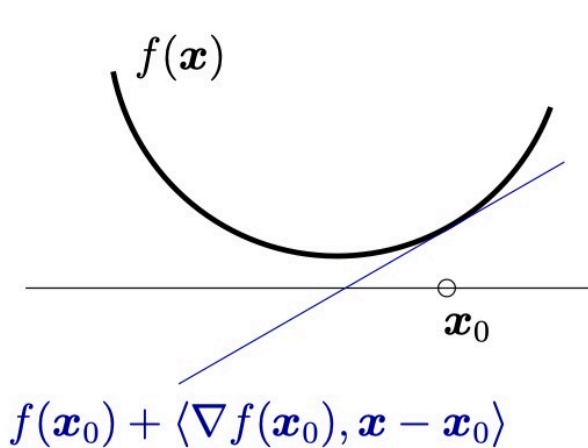
Fact: the constrained soft-margin SVM is equivalent to the following unconstrained formulation:

$$\min_{\mathbf{w} \in \mathbb{R}^d, b} \frac{1}{n} \sum_{i=1}^n \underbrace{\max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b))}_{H(\boldsymbol{\theta}; \mathbf{x}_i, y_i)} + \lambda \|\mathbf{w}\|_2^2$$

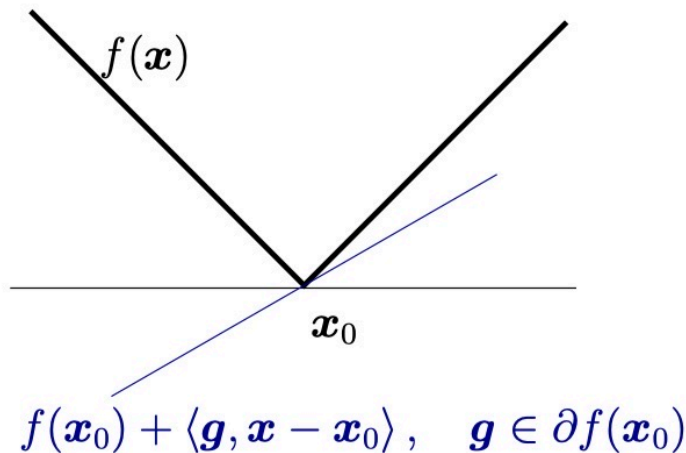
- $\lambda > 0$ is the penalty (hyperparameter)
 - $\lambda > 0$ determines the trade-off between increasing the margin and ensuring that $\mathbf{x}_i$ lie on the correct side of the margin
 - if data is linearly separable, then we can choose a sufficiently small λ to let the soft-margin SVM works as the same to hard-margin SVM

How to solve the soft-margin SVM?

- **Challenge:** the hinge loss is *not* differentiable



differentiable



nondifferentiable

Subgradient Method

- **Subgradient.** Let $f : \mathbb{R}^d \mapsto \mathbb{R}$ be convex. A *subgradient* of f at $\mathbf{x}_0$ is any $\mathbf{u}$ satisfying

$$f(\mathbf{x}) \geq f(\mathbf{x}_0) + \langle \mathbf{u}, \mathbf{x} - \mathbf{x}_0 \rangle, \quad \forall \mathbf{x}.$$

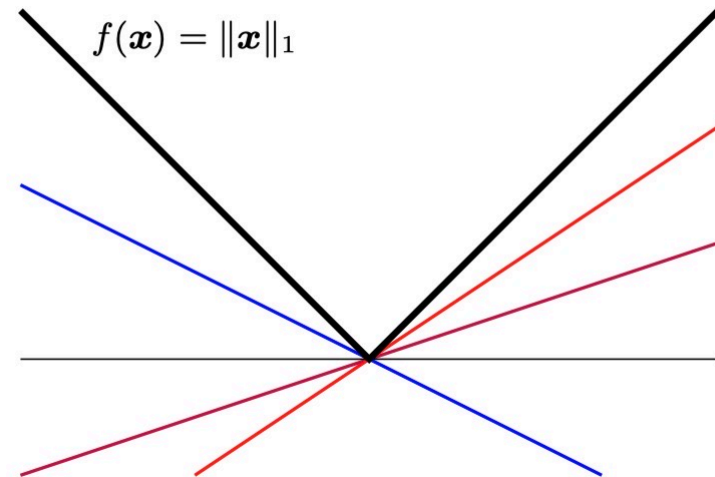
- **Subdifferential.** It is the set of *all* subgradients of f at $\mathbf{x}_0$

$$\partial f(\mathbf{x}_0) := \{ \mathbf{u} \mid f(\mathbf{x}) \geq f(\mathbf{x}_0) + \langle \mathbf{u}, \mathbf{x} - \mathbf{x}_0 \rangle, \quad \forall \mathbf{x} \in \mathbb{R}^d \}.$$

If f is differentiable at $\mathbf{x}_0$, then $\partial f(\mathbf{x}) = \{\nabla f(\mathbf{x}_0)\}$

Example: $f(x) = |x|$

$$\partial f(x) = \begin{cases} \{1\} & x > 0, \\ [-1, 1] & x = 0, \\ \{-1\} & x < 0. \end{cases}$$



Subgradient Method for Soft-Margin SVM

$$\min_{\mathbf{w} \in \mathbb{R}^d, b} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)) + \lambda \|\mathbf{w}\|_2^2$$

Subgradient method:

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - \tau_k \cdot \mathbf{d}^k, \quad \forall \mathbf{d}^k \in \partial \mathcal{L}(\boldsymbol{\theta}^k).$$

- The stepsize $\tau_k > 0$ is often chosen as diminishing, for example, $\tau_k = \frac{c}{k}$
- **Issue:** subgradient method is usually **slow** in convergence

Further Readings & References

- [The Elements of Statistical Learning: Data Mining, Inference, and Prediction](#), by Trevor Hastie, Robert Tibshirani, and Jerome Friedman (Chapter 12)
- [Foundations of Machine Learning](#), by Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. (Chapter 4)
- *Subgradient Methods, Notes for EE364b, Stanford University*. Stephen Boyd.
https://stanford.edu/class/ee364b/lectures/subgrad_method_notes.pdf